

JavaScript Error Handling – Complete Answers

Answers for Tasks 1–15. Tasks 2, 5, 8, 11, and 14 repeat earlier questions, so the same correct solution applies.

Tasks 1 & 2 – Validate Spotify URL

```
function validateSpotifyURL(url) {
  if (!url.startsWith("https://open.spotify.com/")) {
    throw new Error("Invalid Spotify URL");
  }
  return "Valid Spotify URL";
}

try {
  console.log(validateSpotifyURL("https://open.spotify.com/playlist/123"));
  console.log(validateSpotifyURL("https://example.com/"));
} catch (error) {
  console.log(error.message);
}
```

Checks whether the URL begins with the required Spotify base URL. If it does not, an Error object is thrown with the requested message.

Task 3 – Validate Username

```
function validateUsername(username) {
  if (!/^[A-Za-z0-9]+$/.test(username)) {
    throw new Error("Invalid Username");
  }
  return "Valid Username";
}

try {
  console.log(validateUsername("John123"));
  console.log(validateUsername("John@123"));
} catch (error) {
  console.log(error.message);
}
```

The regular expression allows only uppercase/lowercase letters and digits. Any other character causes a custom Error to be thrown.

Tasks 4 & 5 – Parse Flipkart Product JSON

```
const productData = '{"name":"Shoes","price":999}';

try {
  const product = JSON.parse(productData);
  console.log(product);
} catch (error) {
  console.log("Invalid product data");
} finally {
  console.log("Parsing attempt finished");
}
```

JSON.parse() is placed inside try. Invalid JSON is handled by catch, while finally always runs whether parsing succeeds or fails.

Task 6 – Check Playlist URL

```
function checkPlaylistURL(url) {
  if (!url.startsWith("https://open.spotify.com/")) {
    throw new Error("Invalid Spotify URL");
  }
  return "Valid Spotify URL";
}
```

```

try {
  console.log(checkPlaylistURL("https://example.com/playlist/123"));
} catch (error) {
  console.log(error.message);
} finally {
  console.log("Validation complete");
}

```

The catch block displays the custom error message, and finally logs the required completion message.

Tasks 7 & 8 – Check Payment Amount

```

function checkPaymentAmount(amount) {
  if (amount <= 0) {
    throw new Error("Amount must be positive");
  }
  return "Payment accepted";
}

try {
  console.log(checkPaymentAmount(500));
  console.log(checkPaymentAmount(0));
} catch (error) {
  console.log(error.message);
}

```

Amounts of zero or below throw an Error. Positive amounts return 'Payment accepted'.

Task 9 – Get Zomato Order Total

```

function getOrderTotal(items) {
  let total = 0;

  for (const item of items) {
    if (item.price < 0) {
      throw new Error("Invalid item price");
    }
    total += item.price;
  }

  return total;
}

const items = [
  { name: "Pizza", price: 299 },
  { name: "Burger", price: 199 },
  { name: "Fries", price: 99 }
];

try {
  console.log("Order total:", getOrderTotal(items));
} catch (error) {
  console.log("Sorry, your order contains an invalid item price.");
}

```

The function checks every item before adding its price. A negative price throws an Error, which is converted into a friendly message by catch.

Tasks 10 & 11 – Handle Undefined User

```

let user;

try {
  console.log(user.name);
} catch (error) {
  console.log("User data not found");
}

```

Because user is undefined, accessing user.name throws a TypeError. The catch block prevents the program from crashing and displays the requested friendly message.

Task 12 – Fix Error Object Handling

```
function formatFollowers(count) {  
  try {  
    if (count < 0) {  
      throw new Error("Negative count");  
    }  
    return count + " followers";  
  } catch (e) {  
    return e.message;  
  }  
}  
  
console.log(formatFollowers(100)); // 100 followers  
console.log(formatFollowers(-5)); // Negative count
```

The bug was caused by throwing a string. Strings do not have the Error object's message property. Throwing new Error(...) provides e.message.

Task 13 – Explain a Browser Stack Trace

```
// Example code that produces a stack trace:  
function validateSpotifyURL(url) {  
  if (!url.startsWith("https://open.spotify.com/")) {  
    throw new Error("Invalid Spotify URL");  
  }  
}  
  
validateSpotifyURL("https://example.com");
```

Example console output (your browser's exact file names and line numbers will differ):

```
Error: Invalid Spotify URL  
at validateSpotifyURL (script.js:4:11)  
at script.js:8:1
```

Explanation:

- "Error: Invalid Spotify URL" is the error type and message.
- "validateSpotifyURL (script.js:4:11)" identifies the function and the line/column where the error was thrown.
- "script.js:8:1" shows the code that called validateSpotifyURL.
- The stack trace is the chain of function calls that led to the error.

Summary in your own words:

"The error happened because the URL did not start with the required Spotify URL. The stack trace shows exactly where the error was thrown and which code called that function."

Task 14 – Explain Another Stack Trace

```
// Example:  
const product = JSON.parse('{ "name": "Shoes", }');  
  
try {  
  JSON.parse('{ "name": "Shoes", }');  
} catch (error) {  
  console.error(error);  
}
```

Example console output:

```
SyntaxError: Expected double-quoted property name in JSON at position ...  
at JSON.parse ()  
at script.js:4:8
```

Explanation:

- SyntaxError means the JSON text is not valid JSON syntax.
- JSON.parse is the operation that detected the problem.
- script.js:4:8 identifies the line and column where JSON.parse was called.
- The exact wording and position can vary by browser.

Summary in your own words:

"The JSON string contains invalid syntax, so JSON.parse failed. The stack trace points to the JSON.parse call that caused the error."

Task 15 – Stack Trace + Comment Above Code

```
// Summary: The function throws a custom Error when the URL is invalid.  
// The stack trace points to the throw statement inside validateSpotifyURL  
// and then shows the line where validateSpotifyURL was called.
```

```
function validateSpotifyURL(url) {  
  if (!url.startsWith("https://open.spotify.com/")) {  
    throw new Error("Invalid Spotify URL");  
  }  
}  
  
try {  
  validateSpotifyURL("https://example.com");  
} catch (error) {  
  console.error(error.stack);  
}
```

Example stack trace:

```
Error: Invalid Spotify URL  
at validateSpotifyURL (script.js:7:11)  
at script.js:12:3
```

The first relevant line tells you that the error was created at the throw statement inside validateSpotifyURL. The next line shows that the function was called from the try block. Your actual browser will show its own file name and line numbers.

Important note for Tasks 13–15

The examples above demonstrate the expected format. If your assignment specifically requires a stack trace copied from your own browser console, run the supplied code in your browser, copy the actual console stack trace, and use that trace in your submission. Line numbers and browser wording may differ.